

The logo of Gazi University is a circular seal. It features the university's name in Turkish, "GAZİ ÜNİVERSİTESİ", around the top inner edge and the year "1926" at the bottom. In the center is a stylized signature of "Gazi".

GAZİ UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

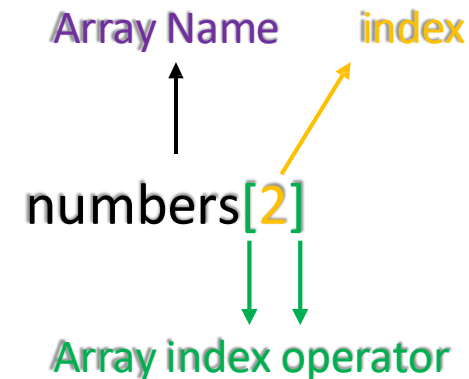
IE104-COMPUTER PROGRAMMING I

WEEK 10: ARRAYS AND COLLECTIONS

Arrays

- A set of objects that are the same type and are consecutively placed in the memory.
- All elements in an array are accessed by the same name.
- The name of all elements in the array is common. The distinguishing aspect between the elements is their location in the computer memory.
- A 5-element integer array is placed in the memory as follows:

numbers[]
25
13
-5
0
4563



Arrays

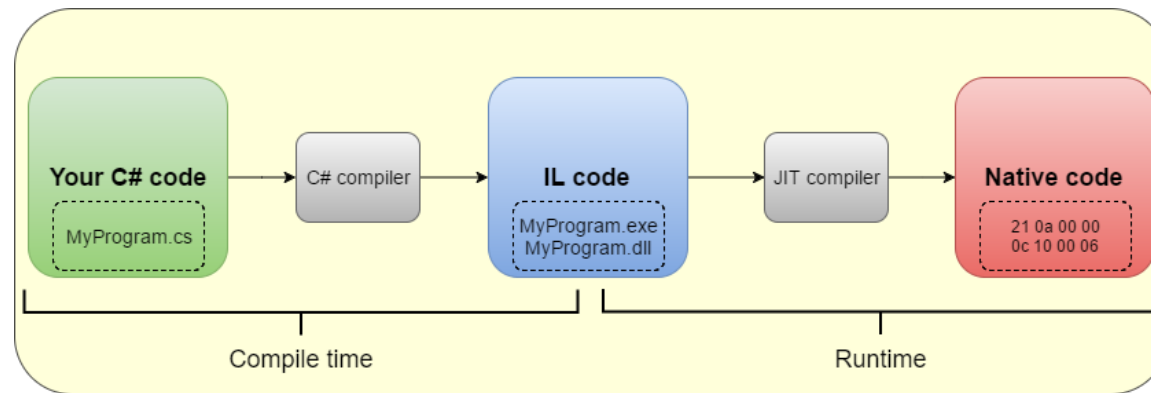
- Arrays in C# are designed as a separate type.
- All arrays are derived from the System.Array class.
- When an array is defined, an object of type System.Array is defined.
- A compilation error does not occur when the array exceeds the bounds of the index, but an error occurs at runtime.
- Array definition can be done in 2 ways:
 - **int [] array = new int[19]**
 - OR
 - **int [] array;**
 - **array=new int [19];**

Arrays

- With the **new** keyword, each element of the array is given the initial value
- for basic data types.
- The initial value is; null for reference types, 0 for numeric types, and false for bool types.
- It is also possible to define arrays as follows:
 - **string[] array1={ “Computer”, “Engineering”, “Department” };**
 - **int [] nums ={ 5, 9, 12,5 6, 23 };**
 - **float [] array3 = { 0.8f, 2.39f, 3.24f, 2.3f, 5.2f };**

Arrays

- In C++ and in many C contexts, array sizes must be specified before use.
- Accordingly, memory for the array is allocated on the basis of that specified size.
- In C#, however, arrays are reference types, and their length is established at runtime.
- Once the length of an array has been established, it cannot be modified.
- Therefore, an existing array cannot itself be dynamically resized.



Arrays

TRY THIS:

```
class Program // Gazi IE Array Exercise
{
    0 başvuru
    static void Main(string[] args)
    {
        Console.Write("Enter the size of the array:");
        int arrsize = int.Parse(Console.ReadLine());

        bool[] array = new bool[arrsize];

        foreach (var item in array)
        {
            Console.WriteLine(item);
        }
    }
}
```

Arrays

- Try this:

```
class Program // Gazi IE Array Exercise
{
    0 başvuru
    static void Main(string[] args)
    {
        int k = 0;
        int[] array = new int[10];

        k = array.Length;

        Console.WriteLine("Size of Array:{0}",k);
    }
}
```

The Output is → What is the output ?

Arrays – Random Value

- Assigning a random value to an array;
- With the Random class is defined in the System namespace.

```
Random rasgele_olustur = new Random();  
int[] rasgele_array = new int[3];  
// Following generates random numbers from 2 to 10, Mat: [2,10)  
rasgele_array [0] = rasgele_olustur.Next(2,10);  
// between 0 and 50, Mathematically: [0,49)  
rasgele_array[1] = rasgele_olustur.Next(50);  
//random integer value from 0 to Int32.MaxValue-1. [0,232)  
rasgele_array[2] = rasgele_olustur.Next();
```


Arrays – Random Value

- Open your compiler and write the main function.
- Attention! You have to add a class and a namespace outside of this code to make it work. Please recall your prior knowledge on this.

```
static void Main(string[] args)
{
    const int collectionSize = 20;
    Random randomGenerator = new Random();
    int[] quantityArray = new int[collectionSize];
    char[] characterArray = new char[collectionSize];

    for (int i = 0; i < collectionSize; i++)
    {
        quantityArray[i] = randomGenerator.Next(1, 51);
        characterArray[i] = (char)randomGenerator.Next(20, 126);
    }

    // Visualize the data as a histogram-style output
    for (int i = 0; i < collectionSize; i++)
    {
        int currentQuantity = quantityArray[i];
        char currentSymbol = characterArray[i];
        Console.Write($"{i,2}. value {currentQuantity,2} --> ");

        for (int j = 0; j < currentQuantity; j++)
        {
            Console.Write(currentSymbol);
        }

        Console.WriteLine();
    }
}
```

Multidimensional Arrays

- There are two types of multidimensional arrays.
- **Regular** arrays (matrices) with an equal number of elements in each dimension and **jagged** arrays with a different number of elements in each dimension.
- For multidimensional array definition, comma (,) is added in [] during definition as much as the size to be given.
- Written as “[, ,]”.

Initial values of an empty array is shown here

int → 0
double → 0.0
float → 0
bool → false
char → '\0' (null character)
string → null
any reference type → null
structs → all fields set to their default values

Multidimensional Arrays

We use multidimensional arrays to store couple of values together.

- `int [,] katli_arr = {{1,2},{3,4},{5,6}};`
- 3 Rows and 2 columns elements of a 3x2 array defined as:

`katli_arr[0,0]`

`katli_arr[0,1]`

`katli_arr[1,0]`

`katli_arr[1,1]`

`katli_arr[2,0]`

`katli_arr[2,1]`

katli_arr

<code>katli_arr[0,0]</code>	<code>katli_arr[0,1]</code>
<code>katli_arr[1,0]</code>	<code>katli_arr[1,1]</code>
<code>katli_arr[2,0]</code>	<code>katli_arr[2,1]</code>

Values


array

1	2
3	4
5	6

Other samples:

- `int[,] multi_array=new int[2,3];`
 - `multi_array[0,0]=1;`
 - `multi_array[2,1]=7;`

Multidimensional Arrays

TRY THIS:

```
1  using System;
2
3  namespace multi_array
4  {
5      class MultiDimensionalArrayExample
6      {
7          static void Main(string[] args)
8          {
9              int[,] matrix =
10             {
11                 { 1, 2 },
12                 { 3, 4 },
13                 { 5, 6 }
14             };
15
16             for (int row = 0; row < 3; row++)
17             {
18                 for (int column = 0; column < 2; column++)
19                 {
20                     Console.Write("{0,5}", matrix[row, column]);
21                 }
22                 Console.WriteLine();
23             }
24         }
25     }
26 }
```

Multidimensional Arrays

- Try this:

```
static void Main(string[] args)
{
    int[,] multiarray = new int[4, 4];
    int k = 1;
    for (int i = 0; i < 4; i++)
        for (int j = i; j < 4; j++)
        {
            multiarray[i, j] = k;
            multiarray[j, i] = k;
            k++;
        }
    k = 1;
    foreach (int x in multiarray)
    {
        if (k % 4 == 0)
        {
            Console.Write(x + " ");
            Console.WriteLine();
        }
        else
            Console.Write(x + " ");
        k++;
    }
}
```

The Output is below:

1	2	3	4
2	5	6	7
3	6	8	9
4	7	9	10

System.Array Class

- Array objects are derived from the System.Array class.
- Therefore, they contain some properties and methods of this class.

Features	Description
IsFixedSize	Returns whether the array has a fixed number of elements. (Boolean)
IsReadOnly	Returns whether the elements of array are read only.
Length	Returns the number of array elements.
Rank	Returns the array dimension.

System.Array Class – Methods

Methods	Description
BinarySearch	Searches a value in one dimensional array due to the binary search algorithm.
Clear	Assigns all the elements of array to the default value.
Copy	Copies some part of an array to another array.
CopyTo	Copies the specified part of an array to the other array.
GetLength	Returns the number of array elements.
GetValue	Returns the value of array element.
Reverse	Reverse the array.
SetValue	Assigns a value to the array element.
Sort	Sorts the one dimensional array.
CreateInstance	Creates a new element of an array.

Get Length – Set/Get Value

- `int[ ] array = { 1, 22, 34 };`
 - `Console.WriteLine(array.GetValue(2));` // Returns 34.
 - `Console.WriteLine(array[2]);` // Returns 34.
- `int v = array.GetLength(0);` //row
- `int y = array.GetLength(1);` //column

SetValue: Method is used to change the value of the element at the specified indexes.

- `array.SetValue(20,0);` //(value,1. index)
- `array.SetValue(20,0,0);` //(value,1.index,2.index)

GetValue: Method returns the value of the element at the specified indexes.

Create Instance

- An array object can be created with the CreateInstance method as follows:

Array my_fancy_arr=Array.CreateInstance(typeof(int),5);

- In the definition above, an array of type int with 5 elements is created.
- The first parameter of the CreateInstance method should always be **typeof**.
- The typeof() operator allow allocation of integer type in the memory.

For instance:

- Array my_char_arr=Array.CreateInstance(typeof(char),3,5);

• A 2-dimensional array of type char is defined above
And the shape is show on the table at the right with
initial value of \0 for every cell.

char	char	char	char	char
char	char	char	char	char
char	char	char	char	char

Array - Exercise

- Try this recursive for loop example:

```
class Program // Gazi IE - Array Exercise
{
    0 başvuru
    static void Main(string[] args)
    {
        Array myarr = Array.CreateInstance(typeof(int), 5, 4, 3);
        Random r = new Random();

        for (int i = 0; i < myarr.GetLength(0); i++)
        {
            for (int j = 0; j < myarr.GetLength(1); j++)
            {
                for (int k = 0; k < myarr.GetLength(2); k++)
                {
                    myarr.SetValue(r.Next(10, 100), i, j, k);
                    Console.WriteLine("myarr[{0}, {1}, {2}]=3,3",
                                     i, j, k, myarr.GetValue(i, j, k));
                }
            }
        }
    }
}
```

CopyTo

- With the CopyTo method, an entire array can be copied to the desired location of another array.
- The elements of array1 are copied to array2 starting with the 3rd index.

➤ `int[ ] flat_array= {1,2,3,4,5,6,7};`

➤ `int empty_array=new int[10];`

➤ `flat_array.CopyTo(empty_array,3);`

Copy

- The same operation can be performed using the Copy() method.
- Such a use below copies the length of elements from array1 to array2. The Copying process starts from index 0.

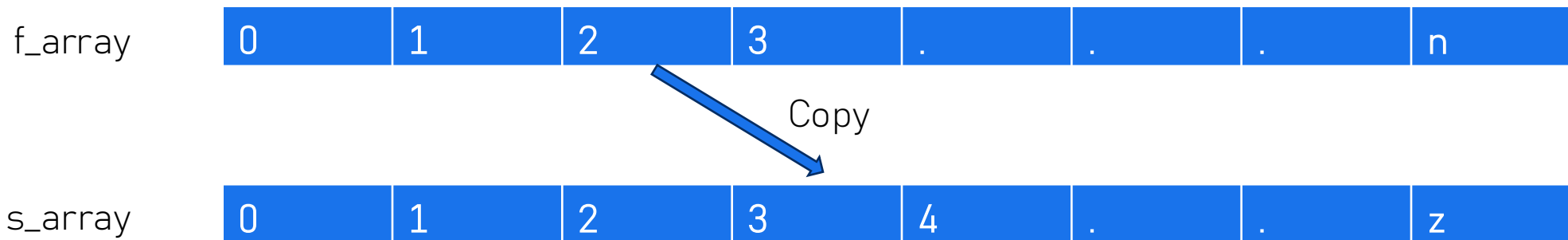
➤ **Copy(Array first_array, Array sec_array, int length);**

- Copies the length after the x Index of array1 to after the y Index of array2.

➤ **Copy(Array first_array, int x, Array sec_array, int y, int length);**

➤ **Array.Copy(f_array, 2, s_array, 3, 1);**

// starting from the 2nd index of f_array, one element is copied to s_array starting from the 3rd index.



Array Sort

- It is used to sort arrays from smallest to largest.
- Basically, the common usage as follows:
 - `Array.Sort(Array);`

```
1 using System;
2 namespace GaziIE.Exercises
3 {
4     class Program
5     {
6         static void Main(string[] args)
7         {
8             string[] studentNames = { "Zeynep", "Berra", "Ramadan", "Mahmoud", "Yusuf" };
9
10            Console.WriteLine("Elements of the Array are:");
11
12            foreach (string name in studentNames)
13            {
14                Console.WriteLine(name);
15            }
16
17            // Performing in-place alphabetical sort
18            Array.Sort(studentNames);
19            // \n\n provides double spacing for visual clarity in the console
20            Console.WriteLine("The Sorted Array as follows:");
21
22            // Displaying the sorted list
23            foreach (string name in studentNames)
24            {
25                Console.WriteLine(name);
26            }
27        }
28    }
29 }
```

Binary Search

- Used for search an object in ordered arrays.
- The Binary Search method has two important usages.

➤ `BinarySearch(Array myarr, object obj);`

❑ This method searches for the object in the array; If the searched object is found, the index is returned. If not, a negative number will be returned.

➤ `BinarySearch(Array myarr, int start, int length, object obj);`

❑ This method, on the other hand, searches only a specific portion of the array defined by starting index and the length, not the whole array.

Binary Search

TRY THIS:

```
1  using System;
2  namespace Exercises
3  {
4      class Program
5      {
6          static void Main(string[] args)
7          {
8              string[] studentNames = { "Zeynep", "Berra", "Ramadan", "Mahmoud", "Yusuf" };
9
10             Console.WriteLine("What name would you like to search?:");
11             string searchName = Console.ReadLine();
12
13             // BinarySearch requires the array to be sorted beforehand to work correctly
14             Array.Sort(studentNames);
15
16             // Searching for the element
17             int searchResultIndex = Array.BinarySearch(studentNames, searchName);
18
19             // Output logic
20             if (searchResultIndex < 0)
21             {
22                 // BinarySearch returns a negative number if the item is not present
23                 Console.WriteLine("The name you have searched is not found.");
24             }
25             else
26             {
27                 // Displaying the specific name and its position in the sorted array
28                 Console.WriteLine("{0} is found at the index of {1}", searchName, searchResultIndex);
29             }
30         }
31     }
32 }
```

THE END



CHECK OUT COURSE WEBSITE: [HTTPS://GAZI-END-MUH.GITHUB.IO](https://GAZI-END-MUH.GITHUB.IO)

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM